



INTERNSHIP MINI PROJECT REPORT

SMART RESUME ANALYZER USING MACHINE LEARNING

Name: Kalaiarashi S B

Department: M Tech(CSE) - II Year / Section A

College : Sri Krishna College of Engineering and Technology

Domain: Data Science

TABLE OF CONTENTS

S.NO	TITLE	Page No
1.	INTRODUCTION	1
2.	OBJECTIVE	1
3.	TECHNOLOGIES USED	2
4.	DATASET DESCRIPTION	3
5.	SYSTEM ARCHITECTURE	4
6.	DATA PREPROCESSING	4
7.	HANDLING MISSING VALUES	5
8.	TEXT CLEANING AND PREPROCESSING	6
9.	EXPLORATORY DATA ANALYSIS(EDA)	7
10.	FEATURE ENGINEERING	8
11.	TF-IDF VECTORIZATION	9
12.	TRAIN-TEST SPLIT	9
13.	RANDOM FOREST REGRESSION MODEL	10
14.	PREDICTION	10
15.	MODEL EVALUATION	11
16.	KEY INSIGHTS	12
17.	ADVANTAGES	12
18.	LIMITATIONS	12
19.	FUTURE ENHANCEMENTS	12
20.	CONCLUSION	13

SMART RESUME ANALYZER USING MACHINE LEARNING

1. INTRODUCTION

Machine Learning and Natural Language Processing (NLP) are transforming modern recruitment systems by automating resume screening and job matching processes. Organizations receive thousands of resumes for different job roles, making manual resume analysis time-consuming and inefficient.

This project presents a Smart Resume Analyzer and Job Match AI system that analyzes resumes, compares candidate skills with job requirements, and predicts resume-job compatibility scores using Machine Learning techniques. The project uses NLP preprocessing, TF-IDF vectorization, and Random Forest Regression for intelligent recruitment analysis.

2. OBJECTIVE

The main objectives of this project are:

- To analyze resume and job description data
- To preprocess and clean textual information
- To identify important candidate skills
- To perform Exploratory Data Analysis (EDA)
- To build a Machine Learning model for job matching prediction
- To automate resume screening using NLP and AI techniques

3. TECHNOLOGIES USED

Technology	Purpose
Python	Main programming language
Pandas	Data preprocessing and analysis
NumPy	Numerical operations
Scikit-learn	Machine Learning implementation
Matplotlib	Data visualization
Seaborn	Statistical visualization
TF-IDF Vectorizer	Text vectorization
Random Forest Regressor	Match score prediction model

4. DATASET DESCRIPTION

The Titanic dataset contains passenger details from the Titanic ship disaster.

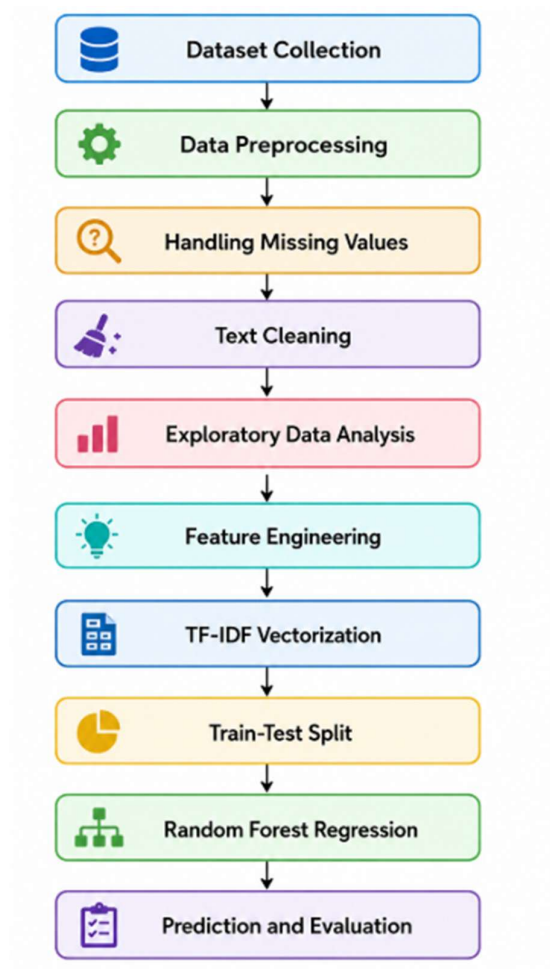
Features Used

Feature	Description
skills	Candidate technical skills
related_skills_in_job	Skills related to the target job
job_position_name	Target job role
educational_requirements	Required qualification
experiencere_requirement	Required experience
skills_required	Required job skills
matched_score	Resume-job compatibility score

Target Variable

Target Variable	Description
matched_score	Numerical score representing compatibility between resume and job description

5. SYSTEM ARCHITECTURE



6. DATA PREPROCESSING

Data preprocessing is one of the most important steps in Machine Learning because real-world datasets often contain noisy text data, missing values, and inconsistent formatting.

The dataset was loaded using Pandas.

```
import pandas as pd

df = pd.read_csv("resume_dataset.csv")
```

Only important columns required for resume-job matching analysis were selected.

```
df = df[
    [
        'skills',
        'related_skills_in_job',
        'job_position_name',
        'educational_requirements',
        'experience_requirement',
        'skills_required',
        'matched_score'
    ]
]
```

7. HANDLING MISSING VALUES

The dataset contained missing values in several categorical columns. Missing values were handled using both row removal and mode imputation techniques.

Rows containing very low percentages of missing values were removed using the `dropna()` method.

```
threshold = len(df) * 0.05

cols_to_drop = df.columns[df.isna().sum() <= threshold]

df.dropna(subset=cols_to_drop, inplace=True)
```

Remaining categorical missing values were replaced using mode imputation.

```
cols_with_missing = df.columns[df.isna().sum() > 0]

for col in cols_with_missing:

    df[col] = df[col].fillna(df[col].mode()[0])
```

Missing values were verified using:

```
df.isna().sum()
```

8. TEXT CLEANING AND PREPROCESSING

Text preprocessing was performed to remove unnecessary symbols, punctuation marks, and inconsistent text formatting.

```
import re

def clean_text(text):

    text = text.lower()

    text = re.sub(r'^a-zA-Z ', ' ', text)

    return text
```

The cleaning function was applied to important text columns.

```
df['skills'] = df['skills'].apply(clean_text)

df['skills_required'] = df['skills_required'].apply(clean_text)
```

9. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis was performed to understand the dataset structure, recruitment trends, and important skills.

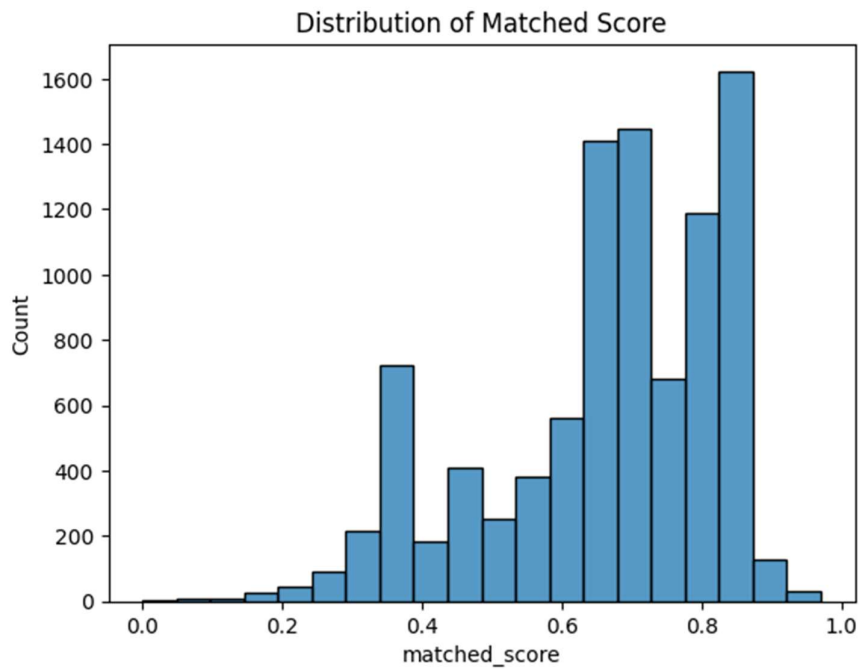
Match Score Distribution

```
sns.histplot(df['matched_score'], bins=20)

plt.title("Distribution of Match Scores")

plt.show()
```

Match Score Distribution Visualization



Top Skills Visualization

```
from collections import Counter

all_skills = []

for item in df['skills']:

    skills = str(item).split()

    all_skills.extend(skills)

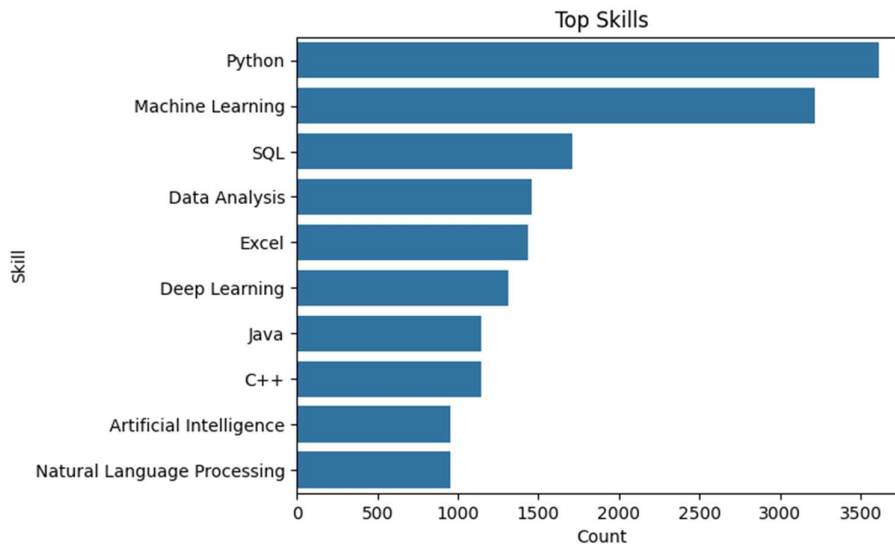
common_skills = Counter(all_skills).most_common(10)
```

```
skill_df = pd.DataFrame(
    common_skills,
    columns=['Skill', 'Count']
)
```

```
sns.barplot(
    x='Count',
    y='Skill',
    data=skill_df
)

plt.title("Top Skills")

plt.show()
```



10. FEATURE ENGINEERING

Important text columns were combined into a single feature column to improve semantic understanding between resumes and job descriptions.

```
df['combined_text'] = (
    df['skills'].astype(str) + " " +
    df['related_skills_in_job'].astype(str) + " " +
    df['skills_required'].astype(str) + " " +
    df['job_position_name'].astype(str)
)
```

11. TF-IDF VECTORIZATION

Machine Learning algorithms cannot directly process textual data. Therefore, TF-IDF Vectorization was used to convert text into numerical feature vectors.

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()

x = tfidf.fit_transform(df['combined_text'])
```

Target variable:

```
y = df['matched_score']
```

12. TRAIN-TEST SPLIT

The dataset was divided into training and testing datasets using an 80:20 ratio.

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(

    x,
    y,
    test_size=0.2,
    random_state=42
)
```

13. RANDOM FOREST REGRESSION MODEL

The Random Forest Regressor algorithm was used for predicting resume-job matching scores.

Random Forest is an ensemble Machine Learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting.

```
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor()

model.fit(x_train, y_train)
```

14. PREDICTION

Predictions were made using the trained Random Forest Regression model.

```
predictions = model.predict(x_test)

print(predictions[:5])
```

Example prediction:

```
resume = "Python SQL Machine Learning"

job = "Python SQL PowerBI"

combined = resume + " " + job

combined = clean_text(combined)

vector = tfidf.transform([combined])

score = model.predict(vector)

print(score)
```

15. MODEL EVALUATION

The model performance was evaluated using regression metrics.

Mean Absolute Error (MAE)

Mean Absolute Error measures the average absolute difference between actual and predicted values.

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, predictions)

print(mae)
```

MAE Achieved:

```
0.078
```

Root Mean Squared Error (RMSE)

RMSE measures prediction error magnitude and gives higher importance to larger errors.

```
from sklearn.metrics import mean_squared_error

import numpy as np

mse = mean_squared_error(y_test, predictions)

rmse = np.sqrt(mse)

print(rmse)
```

RMSE Achieved:

```
0.106
```

The low MAE and RMSE values indicate that the model performs effectively in predicting resume-job compatibility scores.

16. KEY INSIGHTS

- Skills such as Python, Data Analysis, Machine Learning, and SQL appeared frequently in resumes.
- Data Science and Software Engineering were among the most common job positions.
- Higher overlap between candidate skills and required job skills resulted in better matching scores.
- NLP preprocessing significantly improved text consistency and model performance.

17. ADVANTAGES

- Automates resume screening process
- Reduces manual recruitment effort
- Improves candidate-job matching efficiency
- Demonstrates NLP and Machine Learning integration
- Supports intelligent recruitment analytics

18. LIMITATIONS

- Resume text may contain inconsistent formatting
- Skill extraction depends on textual quality
- TF-IDF has limited semantic understanding
- Prediction performance may vary for unseen domains

19. FUTURE ENHANCEMENTS

Future improvements can include:

- BERT and Transformer-based NLP models
- Real-time resume upload system
- Flask or Streamlit deployment
- AI-powered interview recommendation system
- Skill gap analysis dashboard

20. CONCLUSION

This project successfully demonstrated how Machine Learning and NLP techniques can automate resume analysis and job matching tasks.

The system performed data preprocessing, missing value handling, text cleaning, exploratory data analysis, TF-IDF vectorization, and Random Forest Regression to predict resume-job compatibility scores effectively.

The low MAE and RMSE values indicate good prediction performance, making the system suitable for intelligent recruitment and resume screening applications.